Document number: P3006R0
Project: Programming Language C++
Audience: EWGI, EWG, CWG

Antony Polukhin <antoshkka@gmail.com>, <antoshkka@yandex-team.ru>

Date: 2023-10-19

Launder less

> “The problem is that we attempt to solve the simplest questions cleverly, thereby rendering them unusually complex. One should seek the simple solution.”
>
> — *Anton Chekhov*

I. Motivation

```
alignas(T) std::byte storage[sizeof(T)];
::new (&storage) T();
// ...
T *ptr_ = reinterpret_cast<T*>(&storage);  // UB
```

The object that is nested within the `storage` array is not pointer-interconvertible with it [basic.compound] p4. According to the Standard users have to call `std::launder` to avoid UB.

That behavior is surprising for the language users. Moreover, popular compilers do not require `std::launder` call in that place and produce the expected assembly without it.

This proposal attempts to remove UB in that place, standardize existing practice and simplify language usage.

I. More motivating examples

- `boost::optional` uses aligned storage and placement new to store a value. Storing a pointer from placement new may increase the size of an class.
- `utils::FastPimpl` from the userver framework uses byte array and placement new to store a value. Storing a pointer from placement new increases the size of an class.
- HashTable of the ClickHouse project. Storing a pointer from placement new increases the size of an class.
- libcxx in function.h.
- V8 uses reinterpret casts and placement new. Storing the pointer increases object size.
- ...

`std::launder` is required to fix the above cases. However it is counterintuitive, decreases the code readability and requires in-depth knowledge of the Standard to realize that there is an issue from the point of the Standard (but in practice there is no problem!).

III. Wording

Adjust [basic.compound] p4:

```
Two objects a and b are pointer-interconvertible if:
- they are the same object, or
- one is a union object and the other is a non-static data member of that object ([class.union]), or
- one is a standard-layout class object and the other is the first non-static data member of that object or any base class subobject of
- one is an element of an array of std::byte or unsigned char and the other is an object for which the array provides storage, created a
- there exists an object c such that a and c are pointer-interconvertible, and c and b are pointer-interconvertible.
If two objects are pointer-interconvertible, then they have the same address, and it is possible to obtain a pointer to one from a point
[Note 4: An array object and its first element are not pointer-interconvertible, even though they have the same address. — end note]
```

IV. Acknowledgements

Many thanks to Артём Колпаков (Artyom Kolpakov) for asking the question on that topic at STD discussion list and for drawing my attention to the problem. Thanks to Brian Bi for answering the question at STD discussion list.

Thanks to Andrey Erokhin, Timur Doumler, Roman Rusyaev, Mathias Stearn and all the mailing lists members for feedback and help!

Thanks to Andrey Erokhin and Jason Merrill for the wording help!